

# NoSQL

## Introduction et fondements

---

## Rappels sur les bases de données relationnelles et leurs limites

### Les bases de données relationnelles

Les bases de données relationnelles (SGBDR) existent depuis les années 1970. Elles reposent sur quelques principes fondamentaux :

- **Des tables** organisées en lignes et colonnes
- **Un schéma fixe** défini à l'avance (DDL)
- **Le langage SQL** pour interroger et manipuler les données
- **Les contraintes d'intégrité** (clés primaires, clés étrangères, unicité)

Les SGBDR classiques — PostgreSQL, MySQL, Oracle, SQL Server — sont matures, bien documentés et couvrent la majorité des besoins métiers traditionnels.

### Les limites face aux besoins actuels

À partir des années 2000, l'explosion du web et des données a mis en évidence plusieurs limites structurelles :

#### Volumétrie

Un SGBDR classique stocke ses données sur un seul serveur (architecture *scale-up*). Pour gérer des téraoctets voire des pétaoctets, on doit acheter des machines de plus en plus puissantes et coûteuses. Il existe une limite physique et financière à cette approche.

#### Scalabilité horizontale

La scalabilité horizontale consiste à ajouter des machines bon marché plutôt que d'améliorer une seule machine. Les SGBDR relationnels sont difficiles à distribuer horizontalement car :

- Les jointures entre tables nécessitent que toutes les données soient accessibles simultanément
- Les transactions [ACID](#) imposent une coordination forte entre nœuds
- Le partitionnement des données (sharding) est complexe à mettre en œuvre

#### Données non structurées

Les SGBDR imposent un schéma rigide. Or de nombreuses données modernes résistent à cette contrainte :

- Un produit e-commerce peut avoir des attributs très variables selon sa catégorie
- Les données issues de capteurs IoT n'ont pas toutes le même format
- Les contenus générés par les utilisateurs (posts, commentaires, métadonnées) évoluent constamment

Modifier le schéma d'une table de plusieurs centaines de millions de lignes est une opération coûteuse et risquée.

---

## Contexte d'émergence du NoSQL : Big Data et applications web à grande échelle

### L'essor du Big Data

Le terme **Big Data** désigne des volumes de données trop grands, trop rapides ou trop variés pour être traités par les outils traditionnels. On le caractérise par les **3 V** :

| Dimension       | Description                            | Exemple                             |
|-----------------|----------------------------------------|-------------------------------------|
| <b>Volume</b>   | Quantité de données générées           | Des milliards de tweets par jour    |
| <b>Vélocité</b> | Vitesse de génération et de traitement | Flux temps réel d'un réseau social  |
| <b>Variété</b>  | Hétérogénéité des formats              | Texte, images, JSON, logs, capteurs |

### Les pionniers du NoSQL

Face à ces défis, les grands acteurs du web ont développé leurs propres solutions :

- **Google** publie en 2003 le papier *MapReduce* et en 2006 *Bigtable*, une base de données orientée colonnes pour indexer le web
- **Amazon** publie en 2007 *Dynamo*, une base clé-valeur hautement disponible pour son infrastructure e-commerce
- **Facebook** développe *Cassandra* en 2008 pour gérer les boîtes de messages de ses utilisateurs

Ces solutions sacrifient certaines garanties des SGBDR (notamment la cohérence stricte) pour gagner en disponibilité et en capacité de distribution.

Le terme **NoSQL** (*Not Only SQL*) est popularisé en 2009 pour désigner cet ensemble de technologies alternatives.

### Applications web modernes

Les applications web à grande échelle ont des exigences particulières :

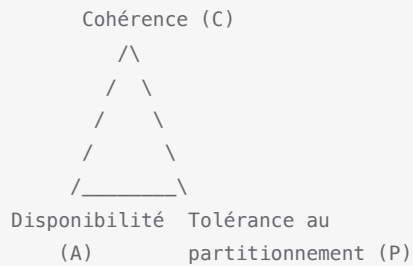
- **Haute disponibilité** : un temps d'arrêt coûte de l'argent (Amazon estime qu'une seconde d'indisponibilité coûte plusieurs milliers de dollars)
- **Latence faible** : les utilisateurs abandonnent une page qui met plus de 3 secondes à charger
- **Montée en charge élastique** : le trafic peut multiplier par 10 lors d'un événement (soldes, lancement de produit)

---

## Le théorème CAP et le modèle BASE vs ACID

### Le théorème CAP

Formulé par Eric Brewer en 2000, le **théorème CAP** (aussi appelé théorème de Brewer) énonce qu'un système distribué ne peut garantir simultanément que deux des trois propriétés suivantes :



- **C — Consistency (Cohérence)** : tous les nœuds voient les mêmes données au même moment. Après une écriture, toute lecture renvoie la valeur la plus récente.
- **A — Availability (Disponibilité)** : le système répond toujours aux requêtes, même si certains nœuds sont défectueux.
- **P — Partition tolerance (Tolérance au partitionnement)** : le système continue de fonctionner même si des messages sont perdus ou si des nœuds sont déconnectés.

Dans un réseau réel, les partitions réseau sont inévitables. Un système distribué doit donc choisir entre **CP** ou **AP** :

| Choix     | Exemples                               | Compromis                           |
|-----------|----------------------------------------|-------------------------------------|
| <b>CP</b> | HBase, Zookeeper, MongoDB (par défaut) | Refuse de répondre si non cohérent  |
| <b>AP</b> | CouchDB, Cassandra, DynamoDB           | Peut renvoyer des données obsolètes |
| <b>CA</b> | SGBDR classiques (non distribués)      | Ne tolère pas les partitions        |

**Note** : Le théorème CAP a été affiné depuis sa formulation initiale. En pratique, il s'agit d'un continuum de compromis plutôt que d'un choix binaire.

## ACID vs BASE

**ACID** est le modèle de cohérence des bases de données relationnelles :

- **Atomicité** : une transaction réussit entièrement ou échoue entièrement
- **Cohérence** : la base passe d'un état valide à un autre état valide
- **Isolation** : les transactions concurrentes ne s'interfèrent pas
- **Durabilité** : une transaction validée est persistée même en cas de panne

**BASE** est le modèle adopté par de nombreux systèmes NoSQL :

- **Basically Available** : le système garantit la disponibilité (selon le CAP)
- **Soft state** : l'état du système peut changer avec le temps, même sans nouvelles entrées
- **Eventually consistent** : le système *finira* par être cohérent, mais pas nécessairement immédiatement

| ACID            |   | BASE                 |
|-----------------|---|----------------------|
| Cohérence forte | ↔ | Cohérence éventuelle |
| Isolation       | ↔ | Optimistic locking   |

|                    |   |                         |
|--------------------|---|-------------------------|
| Pessimiste         | ↔ | Optimiste               |
| Difficile à scaler | ↔ | Scalabilité horizontale |

---

## Panorama des quatre familles NoSQL

### Bases clé-valeur — Redis

**Principe** : les données sont stockées comme des paires clé → valeur. La clé est une chaîne, la valeur peut être n'importe quel type (chaîne, liste, ensemble, hash...).

```
SET user:42:name "Alice"
GET user:42:name → "Alice"
```

#### Caractéristiques :

- Extrêmement rapide (données en mémoire)
- Structure simple, pas de requêtes complexes
- Idéal pour le cache, les sessions, les files de messages

#### Avantages :

- Latence sub-milliseconde grâce au stockage en mémoire
- Très faible complexité opérationnelle
- Supporte des structures variées (listes, sets, hash, sorted sets)

#### Inconvénients :

- Capacité limitée par la RAM disponible
- Pas de requêtes complexes ni de relations entre données
- Persistance sur disque optionnelle (risque de perte en cas de crash)

**Cas d'usage** : cache applicatif, stockage de sessions, classements en temps réel, pub/sub.

**Redis** est le représentant le plus populaire. Il est souvent utilisé comme couche de cache devant un SGBDR.

---

### Bases orientées documents — MongoDB

**Principe** : les données sont stockées sous forme de **documents** (JSON/BSON). Chaque document peut avoir une structure différente.

```
{
  "_id": "ObjectId('...')",
  "nom": "Alice Martin",
  "age": 28,
  "adresses": [
    { "ville": "Paris", "type": "domicile" },
    { "ville": "Lyon", "type": "travail" }
  ]
}
```

json

#### Caractéristiques :

- Schéma flexible
- Requêtes riches sur les champs et documents imbriqués
- Scalabilité horizontale via le sharding

#### Avantages :

- Schéma évolutif sans migration lourde
- Modèle de données proche des objets métier (JSON natif)
- Bonne expressivité des requêtes (filtres, agrégations, index)

#### Inconvénients :

- Dénormalisation pouvant entraîner de la redondance de données
- Pas de jointures natives performantes entre collections
- La flexibilité du schéma peut devenir un risque sans gouvernance

**Cas d'usage :** catalogues de produits, gestion de contenu, applications avec données hétérogènes.

**MongoDB** est la base documentaire la plus utilisée. Nous l'étudierons en détail dans les parties suivantes.

---

## Bases orientées colonnes — Cassandra

**Principe :** les données sont organisées en colonnes plutôt qu'en lignes. Les colonnes fréquemment consultées ensemble sont regroupées (*column families*).

```
Clé de partition → famille de colonnes
user_id:42 → { name: "Alice", email: "alice@...", created_at: "2024-01-01" }
```

#### Caractéristiques :

- Excellente performance en écriture
- Conçu pour la distribution massive (pas de SPOF)
- Requêtes optimisées pour des patterns d'accès prédéfinis

#### Avantages :

- Scalabilité horizontale quasi linéaire
- Haute disponibilité par réplication multi-datacenter
- Performances en écriture très élevées, même à grande échelle

#### Inconvénients :

- Modèle de données rigide : le schéma dépend des requêtes prévues à l'avance
- Cohérence éventuelle par défaut (modèle BASE)
- Pas de jointures, pas de transactions multi-lignes complexes

**Cas d'usage :** séries temporelles, journaux d'événements, données IoT, analytics à grande échelle.

**Apache Cassandra** (initialement développé par Facebook) est utilisé par Netflix, Apple, Instagram pour des charges massives en écriture.

---

## Bases orientées graphe — Neo4j

**Principe** : les données sont représentées sous forme de **nœuds** (entités) et de **relations** (arêtes), chacun pouvant porter des propriétés.

```
(Alice)-[:CONNAÎT {depuis: 2020}]->(Bob)
(Bob)-[:TRAVAILLE_CHEZ]->(EFREI)
```

### Caractéristiques :

- Exploration efficace des relations complexes
- Traversée de graphes en temps constant
- Langage de requête Cypher (déclaratif)

### Avantages :

- Performances constantes sur les traversées de relations, quelle que soit la taille du graphe
- Modélisation intuitive pour les données fortement connectées
- Cypher est lisible et expressif pour les requêtes relationnelles complexes

### Inconvénients :

- Peu adapté aux données tabulaires ou aux requêtes analytiques massives
- Scalabilité horizontale plus difficile que les autres familles NoSQL
- Courbe d'apprentissage si l'on ne pense pas nativement en graphes

**Cas d'usage** : réseaux sociaux, détection de fraude, moteurs de recommandation, graphes de connaissances.

Exemple: [Three and a half degrees of separation](#)

---

## Tableau récapitulatif

| Famille    | Modèle               | Forces                         | Exemples              |
|------------|----------------------|--------------------------------|-----------------------|
| Clé-valeur | Paires clé → valeur  | Rapidité, simplicité           | Redis, DynamoDB       |
| Document   | Documents JSON/BSON  | Flexibilité, requêtes riches   | MongoDB, CouchDB      |
| Colonne    | Familles de colonnes | Écriture massive, distribution | Cassandra, HBase      |
| Graphe     | Nœuds + relations    | Traversée de relations         | Neo4j, Amazon Neptune |

# MongoDB : concepts et modélisation

## Architecture de MongoDB

### Vue d'ensemble

MongoDB est une base de données orientée documents, open source, développée par MongoDB Inc. depuis 2009. Elle stocke les données sous forme de documents JSON-like regroupés dans des collections.

L'architecture de MongoDB s'articule autour de trois niveaux :

```
Instance MongoDB
├─ Base de données (database)
│   └─ Collection
│       └─ Document
```

### Les bases de données

Une **base de données** MongoDB est un conteneur logique pour des collections. Une instance MongoDB peut héberger plusieurs bases de données indépendantes.

### Les collections

Une **collection** est l'équivalent d'une table en SQL. Elle regroupe des documents généralement de même nature, mais sans imposer de schéma commun.

### Les documents JSON/BSON

Un **document** est l'unité de stockage de MongoDB. Il est représenté en JSON (JavaScript Object Notation) mais stocké en **BSON** (Binary JSON) pour des raisons de performance et de richesse de types.

BSON étend JSON avec des types supplémentaires :

| Type BSON                                        | Description                             |
|--------------------------------------------------|-----------------------------------------|
| <code>ObjectId</code>                            | Identifiant unique sur 12 octets        |
| <code>Date</code>                                | Horodatage (millisecondes depuis epoch) |
| <code>NumberInt</code> / <code>NumberLong</code> | Entiers 32/64 bits                      |
| <code>BinData</code>                             | Données binaires                        |
| <code>Decimal128</code>                          | Nombres décimaux précis                 |

```
{
  "_id": { "$oid": "507f1f77bcf86cd799439011" },
  "nom": "Laptop Pro",
```

json

```
"prix": 1299.99,  
"stock": 42,  
"tags": ["informatique", "portable"],  
"createdAt": { "$date": "2024-01-15T10:30:00Z" }  
}
```

## L'ObjectId

Chaque document possède obligatoirement un champ `_id` qui sert de clé primaire. Par défaut, MongoDB génère automatiquement un **ObjectId** de 12 octets composé de :

<https://www.mongodb.com/docs/manual/reference/method/ObjectId/>

|           |                |          |          |  |
|-----------|----------------|----------|----------|--|
| 507f1f77  | bcf86c         | d799     | 439011   |  |
| 4 octets  | 3 octets       | 2 octets | 3 octets |  |
| timestamp | random-machine | PID      | compteur |  |

Cela garantit l'unicité même dans un environnement distribué, sans coordination centrale. Etant donné que chaque ID intègre un timestamp au début de sa valeur, trier les ID par ordre chronologique permet de trier les documents par ordre de création..

## Schéma flexible : avantages, risques et validation

### Le schéma flexible

Contrairement aux SGBDR, MongoDB n'impose pas de schéma à l'insertion. Deux documents d'une même collection peuvent avoir des structures totalement différentes :

```
// Ces deux documents cohabitent dans la même collection  
db.utilisateurs.insertMany([  
  { nom: "Alice", email: "alice@example.com", age: 28 },  
  { nom: "Bob", email: "bob@example.com", premium: true, avatar: "url..." }  
)
```

javascript

### Avantages

- **Évolution rapide** : on peut ajouter de nouveaux champs sans migration de schéma
- **Données hétérogènes** : idéal quand tous les documents n'ont pas les mêmes attributs
- **Prototypage rapide** : pas de DDL à écrire pour commencer à stocker des données

### Risques

- **Incohérence** : sans discipline, une collection peut devenir un fouillis de structures
- **Bugs silencieux** : une faute de frappe dans un nom de champ ne génère pas d'erreur
- **Requêtes imprévisibles** : un champ manquant dans certains documents peut fausser des agrégations

```
// Attention : ces deux insertions sont valides mais incohérentes  
db.produits.insertOne({ nom: "Clavier", prix: 79.99 }) // faute de frappe !
```

javascript



```
db.produits.insertOne({ nom: "Souris", prix: 29.99 })
```

## Validation de schéma (JSON Schema)

MongoDB permet de définir des règles de validation via **JSON Schema** pour obtenir des garanties sur la structure des documents :

```
db.createCollection("produits", {  
  validator: {  
    $jsonSchema: {  
      bsonType: "object",  
      required: ["nom", "prix", "stock"],  
      properties: {  
        nom: {  
          bsonType: "string",  
          description: "Nom du produit (obligatoire)"  
        },  
        prix: {  
          bsonType: "number",  
          minimum: 0,  
          description: "Prix en euros (positif)"  
        },  
        stock: {  
          bsonType: "int",  
          minimum: 0  
        }  
      }  
    }  
  },  
  validationAction: "error" // ou "warn" pour ne pas bloquer  
})
```

javascript

**Bonne pratique** : définir un schéma de validation même partiel. Il peut évoluer avec

```
db.runCommand({ collMod: "produits", validator: {...} }) .
```

## Evolution des données

Si notre modèle de données évolue (changement de nom, de type, ajout de champs, ...) et que l'on souhaite garantir des données uniformes, il faudra nous même parcourir tous les documents à l'aide d'un script pour transformer la donnée.

Consulter la section [migrations](#) pour plus d'informations

---

## Modélisation documentaire : imbrication vs référencement

C'est la question centrale de la modélisation avec MongoDB. Il n'existe pas de réponse universelle : le choix dépend des cas d'usage.

### L'imbrication (embedding)

Les données liées sont stockées **dans le même document**.

```
// Commande avec ses articles imbriqués
{
  "_id": "ObjectId(...)",
  "date": "2024-03-15",
  "client": { "nom": "Alice", "email": "alice@example.com" },
  "articles": [
    { "produit": "Laptop", "quantite": 1, "prix": 1299.99 },
    { "produit": "Souris", "quantite": 2, "prix": 29.99 }
  ],
  "total": 1359.97
}
```

json

### Quand imbriquer ?

- La relation est de type **1-1** ou **1-peu** (one-to-few)
- Les données imbriquées sont **toujours consultées avec** le document parent
- Les données imbriquées ne sont **pas partagées** entre plusieurs documents parents

**Avantages** : une seule requête suffit, pas de jointure, atomicité naturelle des opérations.

**Limites** : un document MongoDB ne peut dépasser **16 Mo**. Si un tableau peut croître sans limite, l'imbrication est dangereuse.

## Le référencement (referencing)

Les données liées sont dans des **collections séparées**, reliées par un identifiant.

Cela ressemble aux foreign keys et jointures en SQL, toutefois moins performant que ces derniers.

```
// Collection "clients"
{ "_id": "ObjectId('c1')", "nom": "Alice", "email": "alice@example.com" }

// Collection "commandes"
{
  "_id": "ObjectId('o1')",
  "clientId": "ObjectId('c1')", // référence vers le client
  "date": "2024-03-15",
  "total": 1359.97
}
```

json

### Quand référencer ?

- La relation est de type **1-N** ou **N-N** avec N potentiellement grand
- Les données liées sont **accédées indépendamment**
- Les données liées sont **partagées** entre plusieurs documents

**Avantages** : pas de duplication de données, documents de taille maîtrisée.

**Limites** : nécessite plusieurs requêtes (ou un `$lookup` dans l'agrégation), pas d'atomicité entre collections.

## Penser par les requêtes

En **SQL**, on modélise d'abord les entités et leurs relations (Merise, UML), puis on adapte les requêtes au modèle.

En **MongoDB**, la démarche est inversée : **on commence par les requêtes** pour déterminer le modèle optimal.

Questions à se poser avant de modéliser :

1. Quelles sont les opérations les plus fréquentes ? (lecture, écriture, mise à jour)
2. Quelles données sont toujours lues ensemble ? → candidates à l'imbrication
3. Quelle est la cardinalité des relations ? (1-1, 1-N, N-N)
4. Les données imbriquées peuvent-elles croître sans limite ?
5. A-t-on besoin d'accéder aux données imbriquées indépendamment ?

### Exemple : système de blog

Entités : Article, Commentaire, Auteur, Tag

Modèle A — tout imbriqué :

```
{
  "titre": "Introduction à MongoDB",
  "auteur": { "nom": "Alice", "email": "alice@..." },
  "tags": ["nosql", "mongodb"],
  "commentaires": [
    { "auteur": "Bob", "texte": "Super article!", "date": "..." },
    { "auteur": "Charlie", "texte": "Merci!", "date": "..." }
  ]
}
```

json

✓ Simple pour lire un article complet ✗ Les commentaires peuvent être très nombreux (limite 16 Mo) ✗ Impossible de paginer les commentaires efficacement

Modèle B — commentaires séparés :

```
// Collection "articles"
{
  "_id": "ObjectId('a1')",
  "titre": "Introduction à MongoDB",
  "auteur": { "nom": "Alice", "email": "alice@..." },
  "tags": ["nosql", "mongodb"]
}

// Collection "commentaires"
{
  "articleId": "ObjectId('a1')",
  "auteur": "Bob",
  "texte": "Super article!",
```

json

```
"date": "2024-03-15"
}
```

✅ Commentaires paginables ✅ Pas de limite de taille pour les commentaires ✅ Lecture de l'article sans charger tous les commentaires

## Comparaison avec la modélisation relationnelle

### Exemple : commande e-commerce

Modèle relationnel (SQL) :

```
-- 4 tables nécessaires
CREATE TABLE clients (id, nom, email);
CREATE TABLE produits (id, nom, prix);
CREATE TABLE commandes (id, client_id, date, total);
CREATE TABLE lignes_commande (id, commande_id, produit_id, quantite, prix_unitaire);
```

sql

Pour lire une commande complète :

```
SELECT c.*, cl.nom, lc.*, p.nom
FROM commandes c
JOIN clients cl ON c.client_id = cl.id
JOIN lignes_commande lc ON lc.commande_id = c.id
JOIN produits p ON lc.produit_id = p.id
WHERE c.id = 42;
```

sql

Modèle documentaire (MongoDB) :

```
{
  "_id": "ObjectId('...')",
  "date": "2024-03-15",
  "client": {
    "id": "ObjectId('c1')",
    "nom": "Alice Martin",
    "email": "alice@example.com"
  },
  "articles": [
    { "nom": "Laptop Pro", "quantite": 1, "prixUnitaire": 1299.99 },
    { "nom": "Souris", "quantite": 2, "prixUnitaire": 29.99 }
  ],
  "total": 1359.97,
  "statut": "livré"
}
```

json

Pour lire une commande complète :

```
db.commandes.findOne({ _id: ObjectId("...") })
// → une seule requête, tout est là
```

javascript

## Tableau comparatif

| Critère      | SQL (relationnel)          | MongoDB (documentaire)             |
|--------------|----------------------------|------------------------------------|
| Schéma       | Fixe, DDL obligatoire      | Flexible, optionnel                |
| Relations    | Jointures explicites       | Imbrication ou référence           |
| Transactions | ACID natives               | ACID multi-documents (depuis v4.0) |
| Scalabilité  | Verticale (principalement) | Horizontale (sharding natif)       |
| Requêtes     | SQL standardisé            | API propriétaire                   |
| Modélisation | Entités → requêtes         | Requêtes → modèle                  |
| Duplication  | Minimisée (3NF)            | Acceptée si nécessaire             |

# MongoDB : requêtes et manipulation de données

---

## Setup : jeu de données d'exemple

Pour illustrer cette partie, nous travaillons avec une collection `films` :

```
const films = collection("films");
```

javascript

```
films.insertMany([
  {
    titre: "Inception",
    annee: 2010,
    realisateur: "Christopher Nolan",
    genres: ["sci-fi", "thriller", "action"],
    note: 8.8,
    duree: 148,
    acteurs: [
      { nom: "Leonardo DiCaprio", role: "Cobb" },
      { nom: "Joseph Gordon-Levitt", role: "Arthur" }
    ]
  },
  {
    titre: "The Dark Knight",
    annee: 2008,
    realisateur: "Christopher Nolan",
    genres: ["action", "crime", "drame"],
    note: 9.0,
    duree: 152,
    acteurs: [
      { nom: "Christian Bale", role: "Batman" },
      { nom: "Heath Ledger", role: "Joker" }
    ]
  },
])
```

javascript

---

## Opérations CRUD

### Insertion

`insertOne` — insérer un seul document :

```
films.insertOne({
  titre: "Dune",
  annee: 2021,
  realisateur: "Denis Villeneuve",
  genres: ["sci-fi", "aventure"],
  note: 8.0,
```

javascript

```

    duree: 155
  })
  // Retourne : { acknowledged: true, insertedId: ObjectId('...') }

```

**insertMany** — insérer plusieurs documents en une seule opération :

```

films.insertMany([
  { titre: "Blade Runner 2049", annee: 2017, realisateur: "Denis Villeneuve", n
  { titre: "Arrival", annee: 2016, realisateur: "Denis Villeneuve", note: 7.9 }
])
// Retourne : { acknowledged: true, insertedIds: { '0': ObjectId('...'), '1': 0

```

Par défaut, **insertMany** s'arrête au premier document en erreur. Passer **{ ordered: false }** pour continuer malgré les erreurs.

## Lecture avec **find**

**find** — retourne un curseur sur les documents correspondants :

```

// Tous les documents
films.find()

// Avec un filtre
films.find({ realisateur: "Christopher Nolan" })

// find retourne un curseur ; findOne retourne directement le premier document
films.findOne({ titre: "Inception" })

```

## Le curseur retourné par **find()**

**find()** ne retourne pas directement un tableau de documents : il retourne un **curseur**, c'est-à-dire un pointeur vers le jeu de résultats côté serveur. Les documents ne sont transférés au client qu'au fur et à mesure qu'on les consomme.

```

const curseur = films.find({ note: { $gte: 8.5 } })

// Itérer manuellement
while (curseur.hasNext()) {
  console.log(await curseur.next())
}

// Ou consommer tous les résultats d'un coup en tableau
const all = await curseur.toArray()

```

Les méthodes chaînables comme **.sort()**, **.skip()** et **.limit()** configurent le curseur **avant** l'exécution de la requête — elles ne déclenchent pas de communication avec le serveur :

```

films.find().sort({ note: -1 }).skip(0).limit(10)
// La requête n'est exécutée que lorsque les résultats sont consommés

```

Un curseur se ferme automatiquement après 10 minutes d'inactivité côté serveur (timeout par défaut). Pour les traitements longs, utiliser `cursor.noCursorTimeout()` avec précaution.

---

## Mise à jour

**updateOne** — met à jour le premier document correspondant :

```
films.updateOne(
  { titre: "Inception" },          // filtre
  { $set: { note: 8.9, vu: true } } // modification
)
// Retourne : { matchedCount: 1, modifiedCount: 1 }
```

javascript

**updateMany** — met à jour tous les documents correspondants :

```
films.updateMany(
  { realisateur: "Christopher Nolan" },
  { $set: { labelQualite: "masterpiece" } }
)
```

javascript

---

## Suppression

**deleteOne** — supprime le premier document correspondant :

```
films.deleteOne({ titre: "Arrival" })
// Retourne : { deletedCount: 1 }
```

javascript

**deleteMany** — supprime tous les documents correspondants :

```
// Supprimer tous les films avant 2010
films.deleteMany({ annee: { $lt: 2010 } })
```

javascript

Attention : `films.deleteMany({})` supprime tous les documents de la collection.

---

## Opérateurs de requête

### Opérateurs de comparaison

```
// $eq : égal (équivalent à { note: 8.8 })
films.find({ note: { $eq: 8.8 } })

// $ne : différent
films.find({ realisateur: { $ne: "Christopher Nolan" } })

// $gt / $gte : supérieur / supérieur ou égal
films.find({ note: { $gt: 8.5 } })
```

javascript



```
films.find({ annee: { $gte: 2010 } })

// $lt / $lte : inférieur / inférieur ou égal
films.find({ duree: { $lt: 150 } })

// $in : dans une liste de valeurs
films.find({ annee: { $in: [2010, 2014, 2019] } })

// $nin : pas dans la liste
films.find({ realiseur: { $nin: ["Christopher Nolan"] } })
```

## Opérateurs logiques

```

// $and : toutes les conditions (implicite avec plusieurs champs)
films.find({ $and: [{ note: { $gte: 8.5 } }, { annee: { $gt: 2010 } }] })
// Équivalent simplifié :
films.find({ note: { $gte: 8.5 }, annee: { $gt: 2010 } })

// $or : au moins une condition
films.find({ $or: [{ note: { $gte: 9.0 } }, { realiseur: "Bong Joon-ho" } ] })

// $not : inverse une condition
films.find({ note: { $not: { $lt: 8.5 } } })

// $nor : aucune des conditions
films.find({ $nor: [{ genres: "action" }, { annee: { $lt: 2015 } } ] })

```

## Opérateurs sur les tableaux

```

// Chercher un tableau contenant une valeur
films.find({ genres: "sci-fi" })
// MongoDB cherche si "sci-fi" est présent dans le tableau genres

// $all : le tableau doit contenir toutes les valeurs
films.find({ genres: { $all: ["sci-fi", "thriller"] } })

// $size : le tableau a exactement N éléments
films.find({ genres: { $size: 3 } })

// $elemMatch : au moins un élément du tableau satisfait TOUTES les conditions
films.find({
  acteurs: { $elemMatch: { nom: "Leonardo DiCaprio", role: "Cobb" } }
})

```

Distinction importante : `{ acteurs: { nom: "A", role: "B" } }` cherche un document où `acteurs.nom == "A"` ET `acteurs.role == "B"`, mais pas nécessairement sur le même élément du tableau. `$elemMatch` garantit que c'est le même élément.

## Requêtes sur documents imbriqués et notation pointée

Pour interroger des champs dans des sous-documents ou des tableaux, MongoDB utilise la **notation pointée** (dot notation).

```
javascript
// Accès à un champ d'un sous-document
films.find({ "acteurs.nom": "Leonardo DiCaprio" })

// Plusieurs niveaux d'imbrication
// Si le document avait : { meta: { pays: { code: "FR" } } }
films.find({ "meta.pays.code": "FR" })

// Accès à un index précis dans un tableau (0-indexé)
films.find({ "acteurs.0.nom": "Christian Bale" })
// Trouve les films dont le PREMIER acteur est Christian Bale
```

---

## Projections, tri et pagination

### Projections

Une **projection** spécifie les champs à inclure ou exclure dans les résultats :

```
javascript
// Inclure uniquement certains champs (1 = inclure)
films.find({}).project({ titre: 1, note: 1, _id: 0 })
// Retourne : { titre: "Inception", note: 8.8 }

// Exclure certains champs (0 = exclure)
films.find({}).project({ acteurs: 0, _id: 0 })
// Retourne tout sauf acteurs et _id

// Projection sur un tableau : $slice
films.find({}).project({ acteurs: { $slice: 1 } })
// Retourne uniquement le premier acteur du tableau
```

On ne peut pas mélanger inclusions et exclusions dans la même projection (sauf pour `_id`).

---

### Tri avec `.sort()`

```
javascript
// Tri croissant (1) par note
films.find().sort({ note: 1 })

// Tri décroissant (-1) par note
films.find().sort({ note: -1 })

// Tri multicritère : par réalisateur puis par note décroissante
films.find().sort({ réalisateur: 1, note: -1 })
```

---

### Pagination avec `.skip()` et `.limit()`

javascript

```
// Limiter à 2 résultats
films.find().limit(2)

// Sauter les 2 premiers résultats (page 2 avec 2 éléments par page)
films.find().sort({ note: -1 }).skip(2).limit(2)
```

**Attention :** `.skip()` est performant seulement sur de petits volumes. Pour de grandes collections, il vaut mieux utiliser une pagination basée sur un champ indexé (cursor-based pagination) :

javascript

```
// Page suivante basée sur la note du dernier document vu
films.find({ note: { $lt: 8.8 } }).sort({ note: -1 }).limit(2)
```

## Mises à jour partielles

MongoDB offre de nombreux opérateurs pour modifier des documents sans les réécrire entièrement.

### Remplacement vs mise à jour partielle

Sans opérateur, `updateOne` et `updateMany` remplacent entièrement le document (seul `_id` est conservé) :

javascript

```
// DANGER : remplace le document, tous les autres champs sont perdus !
films.updateOne(
  { titre: "Inception" },
  { note: 9.0 } // pas d'opérateur comme $set
)

// Le document devient : { _id: ObjectId('...'), note: 9.0 }
// Le titre, l'année, les acteurs... tout a disparu.
```

Pour modifier uniquement certains champs, il faut utiliser des **opérateurs de mise à jour** (`$set`, `$inc`, etc.) :

javascript

```
// CORRECT : seul le champ "note" est modifié
films.updateOne(
  { titre: "Inception" },
  { $set: { note: 9.0 } }
)
```

Pour remplacer volontairement un document entier, préférer `replaceOne()` qui rend l'intention explicite.

## Opérateurs de base

javascript

```
// $set : définir ou modifier un champ
films.updateOne({ titre: "Inception" }, { $set: { note: 9.0, vu: true } })
```

```
// $unset : supprimer un champ
films.updateOne({ titre: "Inception" }, { $unset: { vu: "" } })

// $inc : incrémenter (ou décrémenter avec une valeur négative)
films.updateOne({ titre: "Inception" }, { $inc: { vues: 1 } })
// Si "vues" n'existe pas, il est créé avec la valeur 1

// $rename : renommer un champ
films.updateMany({}, { $rename: { "note": "rating" } })
```

## Opérateurs sur les tableaux

```
// $push : ajouter un élément à un tableau
films.updateOne(
  { titre: "Inception" },
  { $push: { genres: "science-fiction" } }
)

// $push avec $each : ajouter plusieurs éléments
films.updateOne(
  { titre: "Inception" },
  { $push: { tags: { $each: ["top250", "culte"] } } }
)

// $pull : retirer un élément correspondant à une condition
films.updateOne(
  { titre: "Inception" },
  { $pull: { genres: "science-fiction" } }
)

// $addToSet : ajouter un élément seulement s'il n'existe pas déjà
films.updateOne(
  { titre: "Inception" },
  { $addToSet: { genres: "sci-fi" } }
)
// Si "sci-fi" est déjà dans genres, rien ne se passe
```

javascript

## L'option `upsert`

`upsert: true` crée le document s'il n'existe pas, ou le met à jour s'il existe :

```
films.updateOne(
  { titre: "Oppenheimer" }, // filtre
  { $set: { realisateur: "Nolan", annee: 2023 } }, // modification
  { upsert: true } // crée si absent
)
// Si "Oppenheimer" n'existe pas → insertion
// Si "Oppenheimer" existe → mise à jour
```

javascript

## Tableau récapitulatif des opérateurs

| Opérateur                               | Effet                                     |
|-----------------------------------------|-------------------------------------------|
| <code>\$set</code>                      | Définir/modifier un champ                 |
| <code>\$unset</code>                    | Supprimer un champ                        |
| <code>\$inc</code>                      | Incrémenter un nombre                     |
| <code>\$rename</code>                   | Renommer un champ                         |
| <code>\$push</code>                     | Ajouter au tableau                        |
| <code>\$pull</code>                     | Retirer du tableau (par valeur/condition) |
| <code>\$pop</code>                      | Retirer le premier ou dernier élément     |
| <code>\$addToSet</code>                 | Ajouter au tableau (sans doublon)         |
| <code>\$mul</code>                      | Multiplier un nombre                      |
| <code>\$min</code> / <code>\$max</code> | Mettre à jour si plus petit/grand         |

# MongoDB : indexation

---

## Indexation

### Pourquoi indexer ?

Sans index, MongoDB effectue un **collection scan** : il parcourt chaque document pour trouver les correspondances. Sur une collection de millions de documents, c'est prohibitif.

Un **index** est une structure de données (B-tree) qui permet de trouver rapidement les documents correspondant à un critère, sans parcourir toute la collection.

Analogie : l'index d'un livre permet de trouver la page d'un sujet sans lire le livre entier.

**Compromis** : les index accélèrent les lectures mais ralentissent les écritures (chaque insertion/mise à jour doit maintenir les index). Il ne faut pas indexer tous les champs.

---

### Index simples

```
// Créer un index croissant sur le champ "note"
db.films.createIndex({ note: 1 })

// Index décroissant (utile pour les tris fréquents)
db.films.createIndex({ annee: -1 })

// Lister les index d'une collection
db.films.getIndexes()

// Supprimer un index
db.films.dropIndex("note")
```

javascript

L'index `_id` est créé automatiquement et ne peut pas être supprimé.

---

### Index composés

Un index sur plusieurs champs, utile pour les requêtes multi-critères et les tris.

```
// Index sur realisateur + note
db.films.createIndex({ realisateur: 1, note: -1 })

// Cet index optimise :
db.films.find({ realisateur: "Christopher Nolan" }).sort({ note: -1 })
// Et aussi :
db.films.find({ realisateur: "Christopher Nolan", note: { $gte: 8.5 } })
// Mais PAS :
db.films.find({ note: { $gte: 8.5 } })
// (la règle du préfixe : l'index commence par "realisateur")
```

javascript

**Règle du préfixe** : un index composé `{ a: 1, b: 1, c: 1 }` sert pour les requêtes sur `a`, `a+b` ou `a+b+c`, mais pas sur `b` ou `c` seuls.

---

## Index textuels

Pour les recherches en texte libre sur des champs de type chaîne :

```
javascript
// Créer un index textuel sur titre et synopsis
db.films.createIndex({ titre: "text", synopsis: "text" })

// Recherche plein texte
db.films.find({ $text: { $search: "batman gotham nuit" } })

// Avec un score de pertinence
db.films.find(
  { $text: { $search: "batman" } },
  { score: { $meta: "textScore" } }
).sort({ score: { $meta: "textScore" } })
```

## Index multiclés

MongoDB indexe automatiquement les tableaux. Si le champ indexé est un tableau, un index **multiclé** est créé pour chaque valeur du tableau.

```
javascript
// Index sur le tableau "genres"
db.films.createIndex({ genres: 1 })

// Cet index optimise :
db.films.find({ genres: "sci-fi" })
db.films.find({ genres: { $in: ["sci-fi", "thriller"] } })
```

## Index avec options

```
javascript
// Index unique : interdit les doublons
db.utilisateurs.createIndex({ email: 1 }, { unique: true })

// Index partiel : n'indexe que les documents correspondant au filtre
db.films.createIndex(
  { note: -1 },
  { partialFilterExpression: { note: { $gte: 8.0 } } }
)
// Plus léger : seuls les films bien notés sont indexés

// Index TTL : suppression automatique du document après expiration
db.sessions.createIndex(
  { createdAt: 1 },
  { expireAfterSeconds: 3600 } // supprime les sessions après 1 heure
)
```

## Analyser les performances avec `explain()`

### Utilisation

`explain()` permet de comprendre comment MongoDB exécute une requête : quel index est utilisé, combien de documents sont examinés, etc.

```
db.films.find({ note: { $gte: 8.5 } }).explain("executionStats")
```

javascript

### Interpréter le résultat

Les champs clés à examiner :

```
{
  "executionStats": {
    "nReturned": 3,          // documents retournés
    "totalDocsExamined": 4,  // documents examinés
    "totalKeysExamined": 3,  // entrées d'index parcourues
    "executionTimeMillis": 0 // temps d'exécution
  },
  "winningPlan": {
    "stage": "FETCH",        // ou "COLLSCAN" si pas d'index
    "inputStage": {
      "stage": "IXSCAN",     // Index Scan = index utilisé ✅
      "indexName": "note_-1"
    }
  }
}
```

javascript

### Signaux d'alarme

| Indicateur                                        | Signification                     | Action                |
|---------------------------------------------------|-----------------------------------|-----------------------|
| <code>"stage": "COLLSCAN"</code>                  | Parcours complet de la collection | Créer un index        |
| <code>totalDocsExamined &gt;&gt; nReturned</code> | L'index est peu sélectif          | Revoir l'index        |
| <code>executionTimeMillis</code> élevé            | Requête lente                     | Profiler et optimiser |

### Le profiler de MongoDB

Pour identifier automatiquement les requêtes lentes :

```
// Activer le profiler pour les requêtes > 100ms
db.setProfilingLevel(1, { slowms: 100 })

// Consulter les requêtes lentes enregistrées
db.system.profile.find().sort({ ts: -1 }).limit(5)
```

javascript



---

TODO sharding

# MongoDB : agrégation

---

## Pipeline d'agrégation

### Principe

Le **pipeline d'agrégation** est le mécanisme de MongoDB pour transformer et analyser des données. Contrairement à `find()` qui filtre et retourne des documents, l'agrégation permet de :

- Regrouper des documents
- Calculer des statistiques
- Transformer la structure des données
- Joindre plusieurs collections

Un pipeline est une **séquence d'étapes** (stages) : chaque étape reçoit les documents de l'étape précédente, les transforme, et les transmet à l'étape suivante.

```
db.collection.aggregate([
  { $stage1: { ... } },
  { $stage2: { ... } },
  { $stage3: { ... } }
])
```

javascript

---

## Les étapes principales

### `$match` — filtrer les documents

Équivalent d'un `WHERE` en SQL. Placer `$match` le plus tôt possible dans le pipeline pour réduire le nombre de documents à traiter.

```
db.films.aggregate([
  { $match: { realisateur: "Christopher Nolan", note: { $gte: 8.5 } } }
])
```

javascript

---

### `$group` — regrouper et calculer

Équivalent d'un `GROUP BY` en SQL. Le champ `_id` définit la clé de regroupement.

```
// Nombre de films par réalisateur
db.films.aggregate([
  {
    $group: {
      _id: "$realisateur",      // clé de regroupement
      nbFilms: { $sum: 1 },    // compter les documents
      noteMoyenne: { $avg: "$note" },
      noteMax: { $max: "$note" },
    }
  }
])
```

javascript

```

        noteMin: { $min: "$note" }
      }
    }
  })
  // Résultat :
  // { _id: "Christopher Nolan", nbFilms: 3, noteMoyenne: 8.83, ... }
  // { _id: "Bong Joon-ho", nbFilms: 1, noteMoyenne: 8.6, ... }

```

Opérateurs d'accumulation disponibles dans `$group` :

| Opérateur                                  | Description                                   |
|--------------------------------------------|-----------------------------------------------|
| <code>\$sum</code>                         | Somme (ou compte avec <code>\$sum: 1</code> ) |
| <code>\$avg</code>                         | Moyenne                                       |
| <code>\$min</code> / <code>\$max</code>    | Minimum / Maximum                             |
| <code>\$first</code> / <code>\$last</code> | Premier / dernier document                    |
| <code>\$push</code>                        | Construire un tableau avec les valeurs        |
| <code>\$addToSet</code>                    | Tableau sans doublons                         |

## `$sort` — trier les résultats

```

db.films.aggregate([
  { $group: { _id: "$realisateur", noteMoyenne: { $avg: "$note" } } },
  { $sort: { noteMoyenne: -1 } } // tri décroissant
])

```

javascript

## `$project` — transformer la structure des documents

Équivalent d'un `SELECT` en SQL. Permet d'inclure, exclure, renommer ou calculer des champs.

```

db.films.aggregate([
  {
    $project: {
      _id: 0,
      titre: 1,
      annee: 1,
      // Ajouter un champ calculé
      dureeEnHeures: { $divide: ["$duree", 60] },
      // Renommer un champ
      notationImdb: "$note"
    }
  }
])

```

javascript

## `$unwind` — décomposer un tableau

`$unwind` transforme chaque élément d'un tableau en un document séparé.

javascript

```
// Avant $unwind, un document :
// { titre: "Inception", genres: ["sci-fi", "thriller", "action"] }

db.films.aggregate([
  { $unwind: "$genres" }
])

// Après $unwind, trois documents :
// { titre: "Inception", genres: "sci-fi" }
// { titre: "Inception", genres: "thriller" }
// { titre: "Inception", genres: "action" }
```

Utilisation typique : compter le nombre de films par genre.

javascript

```
db.films.aggregate([
  { $unwind: "$genres" },
  { $group: { _id: "$genres", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
// { _id: "sci-fi", count: 2 }
// { _id: "thriller", count: 2 }
// { _id: "drame", count: 2 }
// ...
```

## **\$lookup** — jointure entre collections

**\$lookup** permet de faire une jointure gauche (LEFT JOIN) avec une autre collection.

javascript

```
// Collection "commandes" : { clientId: ObjectId('c1'), total: 150 }
// Collection "clients" : { _id: ObjectId('c1'), nom: "Alice" }

db.commandes.aggregate([
  {
    $lookup: {
      from: "clients",           // collection cible
      localField: "clientId",    // champ dans la collection source
      foreignField: "_id",       // champ dans la collection cible
      as: "clientInfo"           // nom du champ résultat (tableau)
    }
  }
])
// Résultat :
// {
//   total: 150,
//   clientId: ObjectId('c1'),
//   clientInfo: [{ _id: ObjectId('c1'), nom: "Alice" }]
// }
```

**\$lookup** retourne toujours un **tableau**. Utiliser **\$unwind** pour obtenir un objet unique si la relation est 1-1.

---

## Exemples concrets

### Statistiques : top des réalisateurs

```
db.films.aggregate([  
  // Filtrer les films bien notés  
  { $match: { note: { $gte: 8.0 } } },  
  // Regrouper par réalisateur  
  {  
    $group: {  
      _id: "$realisateur",  
      nbFilms: { $sum: 1 },  
      noteMoyenne: { $avg: "$note" },  
      titres: { $push: "$titre" }  
    }  
  },  
  // Garder seulement ceux avec au moins 2 films  
  { $match: { nbFilms: { $gte: 2 } } },  
  // Trier par note moyenne décroissante  
  { $sort: { noteMoyenne: -1 } },  
  // Arrondir la note moyenne  
  {  
    $project: {  
      _id: 0,  
      realisateur: "$_id",  
      nbFilms: 1,  
      noteMoyenne: { $round: ["$noteMoyenne", 2] },  
      titres: 1  
    }  
  }  
])
```

javascript

---

### Jointure entre collections : commandes avec détails clients

```
db.commandes.aggregate([  
  { $match: { statut: "livré" } },  
  {  
    $lookup: {  
      from: "clients",  
      localField: "clientId",  
      foreignField: "_id",  
      as: "client"  
    }  
  },  
  { $unwind: "$client" },  
  {  
    $project: {  
      _id: 0,  
      "client.nom": 1,  
      "client.email": 1,  
      total: 1,  
      date: 1  
    }  
  }  
])
```

javascript

```
    },
    { $sort: { date: -1 } },
    { $limit: 10 }
  ])
}
```

## Transformation : rapport mensuel des ventes

```
db.commandes.aggregate([
  {
    $group: {
      _id: {
        annee: { $year: "$date" },
        mois: { $month: "$date" }
      },
      totalVentes: { $sum: "$total" },
      nbCommandes: { $sum: 1 },
      panierMoyen: { $avg: "$total" }
    }
  },
  { $sort: { "_id.annee": 1, "_id.mois": 1 } },
  {
    $project: {
      _id: 0,
      periode: {
        $concat: [
          { $toString: "_id.annee" }, "-",
          { $toString: "_id.mois" }
        ]
      },
      totalVentes: { $round: ["$totalVentes", 2] },
      nbCommandes: 1,
      panierMoyen: { $round: ["$panierMoyen", 2] }
    }
  }
])
```

javascript

# Connexion à MongoDB depuis un serveur

Il existe plusieurs niveaux d'abstraction pour interagir avec MongoDB depuis une application : du driver officiel bas niveau jusqu'aux ODM (Object Document Mapper) qui ajoutent schémas, validation et helpers.

## Le driver officiel MongoDB

Le driver officiel est la couche la plus proche de MongoDB. Tous les autres outils s'appuient dessus.

### Installation

```
npm install mongodb
```

bash

### Connexion et utilisation

```
import { MongoClient } from 'mongodb'

const client = new MongoClient('mongodb://localhost:27017')

await client.connect()

const db = client.db('maBase')
const films = db.collection('films')

// Insérer
await films.insertOne({ titre: 'Inception', note: 8.8 })

// Lire
const film = await films.findOne({ titre: 'Inception' })

// Mettre à jour
await films.updateOne({ titre: 'Inception' }, { $set: { note: 9.0 } })

// Supprimer
await films.deleteOne({ titre: 'Inception' })

await client.close()
```

javascript

### Gestion de la connexion en production

Ouvrir et fermer une connexion à chaque requête est coûteux. En production, on utilise un **pool de connexions** et on partage l'instance `MongoClient` dans toute l'application.

```
// db.js - module singleton
import { MongoClient } from 'mongodb'

const client = new MongoClient(process.env.MONGO_URI)
```

javascript

```
await client.connect()

export const db = client.db('maBase')
```

javascript

```
// Dans une route Express
import { db } from './db.js'

app.get('/films', async (req, res) => {
  const films = await db.collection('films').find().toArray()
  res.json(films)
})
```

## Avantages et limites du driver

| Avantages                                          | Limites                              |
|----------------------------------------------------|--------------------------------------|
| Accès complet à toutes les fonctionnalités MongoDB | Pas de validation de schéma intégrée |
| Performances maximales                             | Code verbeux pour les cas complexes  |
| Aucune couche d'abstraction superflue              | Pas de gestion des relations         |
| Idéal pour les pipelines d'agrégation complexes    | Typage manuel des documents          |

## Solutions JavaScript

### Mongoose — ODM de référence

[Mongoose](#) est l'ODM (Object Document Mapper) le plus utilisé dans l'écosystème Node.js. Il ajoute une couche de **schémas**, de **validation** et de **middleware** au-dessus du driver officiel.

```
npm install mongoose
```

bash

### Définir un schéma et un modèle

```
import mongoose from 'mongoose'

const filmSchema = new mongoose.Schema({
  titre: { type: String, required: true },
  realisateur: { type: String, required: true },
  annee: { type: Number, min: 1888 },
  note: { type: Number, min: 0, max: 10 },
  genres: [String]
})

const Film = mongoose.model('Film', filmSchema)
```

javascript

### Connexion



```
await mongoose.connect('mongodb://localhost:27017/maBase')
```

javascript

## Opérations CRUD

```
// Créer
const film = await Film.create({ titre: 'Parasite', realisateur: 'Bong Joon-ho' })

// Lire
const films = await Film.find({ note: { $gte: 8.0 } }).sort({ note: -1 })
const film = await Film.findById(id)

// Mettre à jour
await Film.findByIdAndUpdate(id, { $set: { note: 8.7 } }, { new: true })

// Supprimer
await Film.findByIdAndDelete(id)
```

javascript

## Fonctionnalités clés de Mongoose

### Validation intégrée

```
const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: [true, 'Email requis'],
    match: [/^\S+@\S+\.\S+$/, 'Email invalide'],
    unique: true
  },
  age: { type: Number, min: [0, 'Age invalide'], max: 120 }
})
```

javascript

### Middleware (hooks)

```
// Avant la sauvegarde : hacher le mot de passe
userSchema.pre('save', async function (next) {
  if (this.isModified('password')) {
    this.password = await bcrypt.hash(this.password, 10)
  }
  next()
})
```

javascript

### Méthodes et virtuals

```
// Méthode d'instance
userSchema.methods.verifyPassword = function (password) {
  return bcrypt.compare(password, this.password)
}

// Virtual (champ calculé, non stocké en base)
filmSchema.virtual('dureeFormatee').get(function () {
```

javascript

```
return `${Math.floor(this.duree / 60)}h${this.duree % 60}min`
})
```

## Population (jointures)

```
const commentaireSchema = new mongoose.Schema({
  texte: String,
  auteur: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
})

// Résoudre la référence automatiquement
const commentaires = await Commentaire.find().populate('auteur', 'nom email')
```

## Prisma — ORM moderne avec support MongoDB

[Prisma](#) est un ORM TypeScript-first qui supporte plusieurs bases de données dont MongoDB. Il génère un client typé à partir d'un fichier de schéma déclaratif.

```
npm install prisma @prisma/client
npx prisma init --datasource-provider mongodb
```

## Schéma Prisma

```
// prisma/schema.prisma
datasource db {
  provider = "mongodb"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model Film {
  id          String   @id @default(auto()) @map("_id") @db.ObjectId
  titre       String
  realisateur String
  note        Float?
  genres      String[]
}
```

## Utilisation

```
import { PrismaClient } from '@prisma/client'

const prisma = new PrismaClient()

// Créer
const film = await prisma.film.create({
  data: { titre: 'Inception', realisateur: 'Christopher Nolan', note: 8.8 }
})
```

```
// Lire avec filtre
const films = await prisma.film.findMany({
  where: { note: { gte: 8.0 } },
  orderBy: { note: 'desc' }
})

// Mettre à jour
await prisma.film.update({
  where: { id: '...' },
  data: { note: 9.0 }
})
```

## Avantages de Prisma

- Typage TypeScript complet généré automatiquement
- Studio visuel intégré ( `prisma studio` )
- Support multi-bases (PostgreSQL, MySQL, MongoDB...)

**Limite avec MongoDB** : Prisma est avant tout un ORM destiné aux bases relationnelles. Il est possible de l'utiliser avec MongoDB mais avec quelques restrictions. Consulter la [documentation officielle](#) pour savoir ce qu'il est possible ou non de faire.

## Comparaison des approches JavaScript

|                      | Driver officiel      | Mongoose                | Prisma                     |
|----------------------|----------------------|-------------------------|----------------------------|
| Niveau d'abstraction | Bas                  | Moyen                   | Élevé                      |
| Schéma               | Aucun                | Défini en JS            | Défini en DSL              |
| TypeScript           | Génériques manuels   | Types partiels          | Génération automatique     |
| Validation           | Manuelle             | Intégrée                | Intégrée                   |
| Aggregation          | Complète             | Complète                | Limitée                    |
| Idéal pour           | Contrôle total, perf | Apps Node.js classiques | Apps TypeScript, multi-BDD |

## Clients dans d'autres langages

MongoDB fournit des drivers officiels pour tous les langages majeurs.

| Langage | Driver officiel                                                                | ODM populaire                                          |
|---------|--------------------------------------------------------------------------------|--------------------------------------------------------|
| Python  | <code>pymongo</code>                                                           | <code>mongoengine</code> , <code>beanie</code> (async) |
| Java    | <code>mongodb-driver-sync</code> / <code>mongodb-driver-reactivestreams</code> | <code>Spring Data MongoDB</code>                       |

| Langage | Driver officiel                       | ODM populaire             |
|---------|---------------------------------------|---------------------------|
| Go      | <code>mongo-driver</code>             | <code>qmgo</code>         |
| PHP     | <code>mongodb</code> (extension PECL) | <code>Doctrine ODM</code> |
| C#/.NET | <code>MongoDB.Driver</code>           | intégré au driver         |
| Ruby    | <code>mongo</code>                    | <code>Mongoid</code>      |
| Rust    | <code>mongodb</code> (crate)          | —                         |

## Exemple Python avec PyMongo

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["maBase"]
films = db["films"]

# Insérer
films.insert_one({"titre": "Parasite", "note": 8.6})

# Lire
for film in films.find({"note": {"$gte": 8.0}}):
    print(film["titre"])
```

python

## Exemple Python avec Beanie (async / FastAPI)

```
from beanie import Document, init_beanie
from motor.motor_asyncio import AsyncIOMotorClient

class Film(Document):
    titre: str
    note: float

    class Settings:
        name = "films"

# Initialisation
client = AsyncIOMotorClient("mongodb://localhost:27017")
await init_beanie(database=client.maBase, document_models=[Film])

# Utilisation
film = await Film.find_one(Film.note >= 8.0)
```

python

# Migrations de schéma avec MongoDB

Dans une base relationnelle, toute modification de structure (ajouter une colonne, renommer un champ) nécessite une migration SQL exécutée sur la base. MongoDB, avec son schéma flexible, fonctionne différemment — mais les migrations restent nécessaires. Une modification de schéma incohérente avec les données existantes ne pourra pas être effectuée, ce qui garantira la cohérence en toute situation.

---

## Le problème des migrations en NoSQL

Puisque MongoDB n'impose pas de schéma, rien n'empêche d'insérer un document `{ nom: "Alice" }` et un autre `{ name: "Bob" }` dans la même collection. Cette flexibilité est un avantage au démarrage, mais devient un problème à mesure que l'application évolue.

### Ce qui se passe sans migration

Imaginons qu'on renomme le champ `note` en `rating` dans le code :

```
// Anciens documents en base :  
{ titre: "Inception", note: 8.8 }  
  
// Nouveaux documents insérés après le changement :  
{ titre: "Parasite", rating: 8.6 }
```

javascript

La collection contient désormais deux formats. Les requêtes sur `rating` ne retournent pas les anciens documents, et inversement.

---

## Stratégies de migration

### 1. Migration eager (immédiate)

On met à jour **tous les documents existants** en une seule opération, avant ou après le déploiement du nouveau code.

```
// Renommer le champ "note" en "rating" sur tous les documents  
db.films.updateMany(  
  { note: { $exists: true } },  
  { $rename: { "note": "rating" } }  
)
```

javascript

```
// Ajouter un champ avec une valeur par défaut  
db.films.updateMany(  
  { disponible: { $exists: false } },  
  { $set: { disponible: true } }  
)
```

javascript

javascript

```
// Supprimer un champ devenu obsolète
db.films.updateMany(
  {},
  { $unset: { ancienChamp: "" } }
)
```

**Avantage** : la base est cohérente immédiatement. **Inconvénient** : peut être lent sur de grandes collections, et bloque potentiellement les opérations pendant la migration.

---

## 2. Migration lazy (progressive)

On ne modifie pas les documents existants. Le code gère les deux formats et migre chaque document **au moment où il est lu ou écrit**.

javascript

```
// Dans le code applicatif
function normaliserFilm(doc) {
  // Compatibilité ascendante : accepter les deux noms de champ
  if (doc.note !== undefined && doc.rating === undefined) {
    doc.rating = doc.note
    delete doc.note
  }
  // Persister la migration
  db.films.updateOne({ _id: doc._id }, {
    $set: { rating: doc.rating },
    $unset: { note: "" }
  })
}
return doc
}
```

**Avantage** : zéro downtime, migration transparente. **Inconvénient** : le code doit gérer plusieurs versions du schéma pendant la période de transition.

---

## 3. Versionnement des documents

On ajoute un champ `schemaVersion` à chaque document pour savoir quel format il suit.

javascript

```
// Document v1
{ _id: ..., schemaVersion: 1, note: 8.8, titre: "Inception" }

// Document v2 (après migration)
{ _id: ..., schemaVersion: 2, rating: 8.8, titre: "Inception" }
```

javascript

```
function lireFilm(doc) {
  switch (doc.schemaVersion) {
    case 1:
      return migrerV1versV2(doc)
    case 2:
    default:
      return doc
  }
}
```

```
}  
}
```

---

## Downtime

Toute migration de donnée implique un problème majeur: le programme déployé en production ne peut être que sur une version: l'ancienne et la nouvelle, donc le programme ne peut supporter que l'ancien ou le nouveau format de données.

Si une modification du schema doit entrainer une modification des données, il y aura alors forcément une periode de temps (plus ou moins longue selon le volume des données) où les données seront incohérentes. Entre le moment où le script de migration est démarré et où il est fini, les serveurs de production doivent être arrêtés, ce qui crée une periode de downtime.

Pour éviter cette periode de downtime, on peut utiliser les techniques de mapping vues précédemment, ce qui exige de maintenir plus de code entre les versions.

---

## Migrations avec Prisma

Prisma adopte une approche différente : il génère et gère des **fichiers de migration** versionés, comme en SQL.

### Flux de travail

1. Modifier le schéma dans `prisma/schema.prisma`
2. Générer la migration
3. Appliquer la migration

### Exemple : renommer un champ

#### Avant

```
model Film {  
  id    String @id @default(auto()) @map("_id") @db.ObjectId  
  note  Float?  
}
```

prisma

#### Après modification du schéma

```
model Film {  
  id      String @id @default(auto()) @map("_id") @db.ObjectId  
  rating  Float?  
}
```

prisma

### Générer et appliquer

```
npx prisma migrate dev --name rename_note_to_rating
```

bash

Prisma génère un fichier de migration dans `prisma/migrations/` et met à jour le client TypeScript.

**Limite importante :** avec MongoDB, Prisma ne génère **pas** de migration de données automatique. Il met à jour le schéma de validation et régénère le client, mais les documents existants en base ne sont pas modifiés. Il faut compléter avec un script de migration de données.

## Migration de données complémentaire

```
import { PrismaClient } from '@prisma/client'
const prisma = new PrismaClient()

// Script à exécuter une seule fois après la migration de schéma
const films = await prisma.$runCommandRaw({
  update: "Film",
  updates: [{
    q: { note: { $exists: true } },
    u: { $rename: { note: "rating" } },
    multi: true
  }]
})
```

typescript

## Bonnes pratiques

Toujours écrire le rollback ( `down` ) pour pouvoir revenir en arrière en cas de problème.

Tester sur une copie des données de production avant d'appliquer en production.

Ne jamais modifier une migration déjà appliquée — créer une nouvelle migration à la place.

**Découpler déploiement et migration :** pour les grandes collections, appliquer la migration avant le déploiement du nouveau code (ou après), jamais simultanément.

**Préférer les migrations additives** quand c'est possible : ajouter un nouveau champ plutôt que renommer l'ancien, supprimer l'ancien champ dans une migration ultérieure une fois que tout le code utilise le nouveau.

|                                              |            |
|----------------------------------------------|------------|
| Étape 1 : ajouter "rating", conserver "note" | → déployer |
| Étape 2 : migrer les données note → rating   | → déployer |
| Étape 3 : supprimer "note"                   | → déployer |

Cette approche en trois temps évite tout downtime et permet un rollback facile à chaque étape.



# Déploiement de bases de données MongoDB

---

## Développement

Dans la phase de développement d'une application, la manière la plus simple de faire tourner une base de données est d'utiliser docker.

[Image officielle docker](#)

Exemple de `docker-compose.yml` :

```
services:
  mongo:
    image: mongo:latest
    ports:
      - "27017:27017"
    volumes:
      - data:/data/db
volumes:
  data:
```

### INFO

Attention, cette configuration suffit pour du développement mais pas pour une base de production !

---

## Production

En production, il est recommandé d'utiliser un orchestrateur de conteneurs comme Kubernetes pour déployer et gérer les bases de données, mais cela demande une maintenance assez conséquente pour maintenir un service fiable (disponibilité, backups, performances...).

La solution la plus simple est d'utiliser les services dits **managed** proposés par les clouds qui garantissent fiabilité, performance et disponibilité sans efforts.

Pour Mongo, la solution la plus simple est d'utiliser [MongoDB Atlas](#), qui propose une offre gratuite pour les petites applications et qui est intégré dans la plupart des clouds (Vercel, Google Cloud...)

---

## Configuration du client

Afin de configurer le serveur et le client, n'hésitez pas à utiliser les variables d'environnement, typiquement pour l'url de connexion au serveur.

```
# .env
MONGODB_URI=mongodb://localhost:27017
```

```
const client = new MongoClient(process.env.MONGODB_URI);
```

js

Attention, pour que les variables d'environnement soient disponibles dans `process.env`, il faut qu'elles soient d'abord lues. C'est souvent le cas avec des packagers comme Next ou Vite. Sans eux, on peut utiliser le paquet NPM `dotenv` pour lire les variables d'environnement.

```
npm i -D dotenv
```

```
import dotenv from 'dotenv';
dotenv.config();
```

js

---

## GUI

Il existe différentes applications pour inspecter une base de données MongoDB. La solution officielle est [MongoDB Compass](#)

Parmi les autres solutions, on retrouvera Prisma Studio, l'inspecteur de IntelliJ,

# Synthèse et choix technologiques

## Tableau comparatif des familles NoSQL

### Vue d'ensemble

| Critère              | Clé-valeur<br>(Redis)    | Document<br>(MongoDB)                | Colonne<br>(Cassandra)   | Graphe (Neo4j)           |
|----------------------|--------------------------|--------------------------------------|--------------------------|--------------------------|
| Modèle de données    | Paires clé → valeur      | Documents JSON/BSON                  | Familles de colonnes     | Nœuds + relations        |
| Schéma               | Aucun                    | Flexible (optionnel)                 | Partiel (colonnes fixes) | Flexible                 |
| Requêtes             | Par clé uniquement       | Riches (champs, imbrication)         | Par clé de partition     | Traversée de graphes     |
| Jointures            | Non                      | Via <code>\$lookup</code> (limitées) | Non                      | Natives et performantes  |
| Cohérence            | Configurable             | CP (configurable)                    | AP (éventuelle)          | ACID                     |
| Scalabilité          | Horizontale (cluster)    | Horizontale (sharding)               | Horizontale (native)     | Verticale principalement |
| Performance lecture  | Excellente (mémoire)     | Bonne                                | Bonne (par clé)          | Excellente (traversées)  |
| Performance écriture | Excellente               | Bonne                                | Excellente               | Bonne                    |
| Cas d'usage idéaux   | Cache, sessions, pub/sub | CMS, e-commerce, IoT                 | Séries temporelles, logs | Réseaux, recommandation  |
| Exemples             | Redis, DynamoDB          | MongoDB, CouchDB                     | Cassandra, HBase         | Neo4j, Amazon Neptune    |

### Cohérence et disponibilité

| Base      | Positionnement CAP            | Cohérence par défaut                  |
|-----------|-------------------------------|---------------------------------------|
| Redis     | CP (mode cluster)             | Forte                                 |
| MongoDB   | CP                            | Forte (lecture depuis le primaire)    |
| Cassandra | AP                            | Éventuelle (configurable par requête) |
| Neo4j     | CA (non distribué nativement) | ACID                                  |

## Scalabilité horizontale



Cassandra est conçue dès le départ pour la distribution. L'ajout d'un nœud augmente linéairement les capacités de lecture et d'écriture, sans point unique de défaillance (SPOF).

## NoSQL vs SQL : quand choisir l'un ou l'autre

### Choisir un SGBDR (SQL) quand...

#### La cohérence des données est critique

- Systèmes bancaires, comptabilité, paye : les transactions ACID garantissent qu'un virement est soit entièrement effectué, soit annulé — jamais dans un état intermédiaire.
- Données médicales : une erreur de cohérence peut avoir des conséquences graves.

#### Le modèle de données est stable et bien défini

- Les entités et leurs relations sont connues à l'avance.
- Le schéma n'évolue pas fréquemment.

#### Les requêtes sont complexes et ad hoc

- Rapports métier avec agrégations sur plusieurs tables
- Analyses exploratoires où les requêtes ne sont pas prédéfinies
- SQL est un langage standardisé, lisible et puissant

#### Les volumes restent raisonnables

- Jusqu'à quelques centaines de millions de lignes, un SGBDR bien configuré (PostgreSQL, MySQL) est souvent suffisant.

### Choisir NoSQL quand...

#### Les volumes sont massifs et la scalabilité horizontale est nécessaire

- Des milliards de documents, des téraoctets de données
- Le coût du scale-up (machines plus puissantes) devient prohibitif
- → Cassandra, MongoDB avec sharding

#### Le schéma est hétérogène ou évolue rapidement

- Catalogues de produits avec des attributs variables par catégorie
- Applications en phase de prototypage rapide
- Données provenant de sources diverses non uniformisées
- → MongoDB

#### Les accès sont simples et la latence doit être minimale

- Cache applicatif : lire/écrire une valeur par clé en quelques microsecondes
- Stockage de sessions utilisateurs
- → Redis

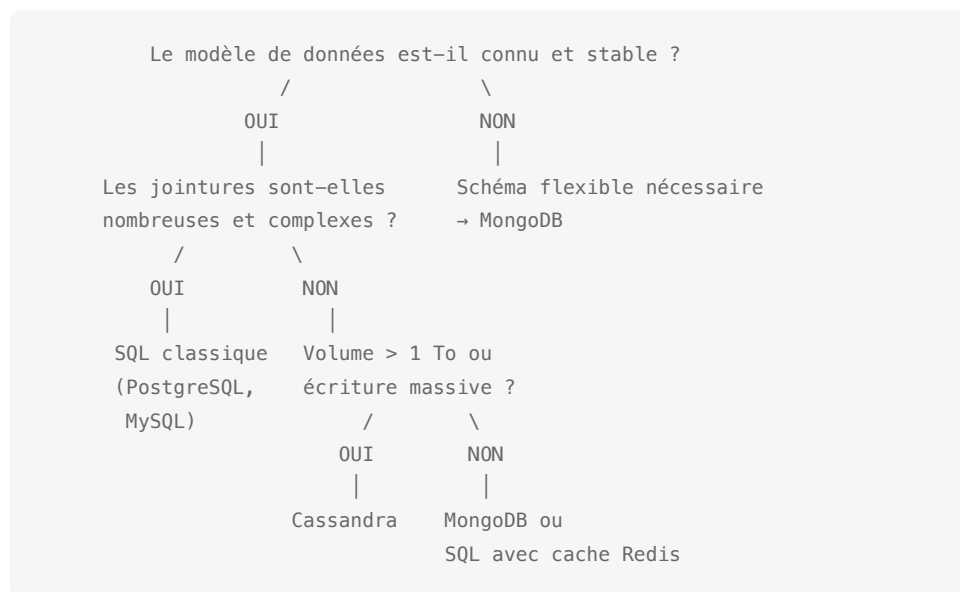
#### Les relations complexes entre données sont au cœur du modèle

- Réseaux sociaux, détection de fraude, recommandation
- → Neo4j

#### Les écritures sont massives et distribuées géographiquement

- Collecte de métriques, journaux d'événements, données IoT
- → Cassandra

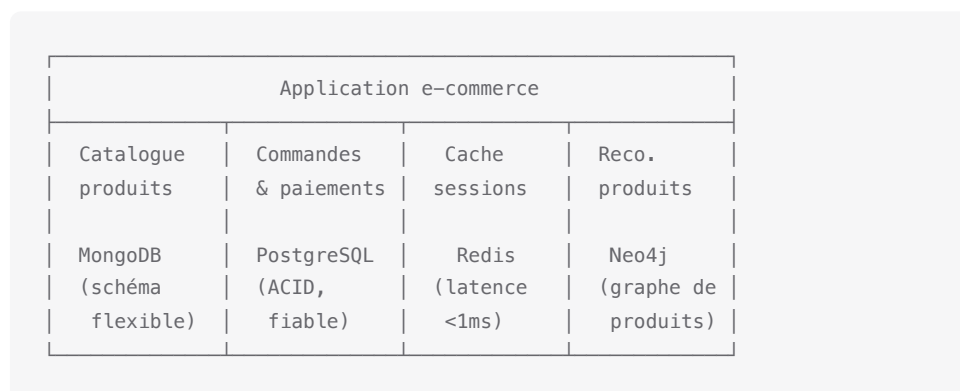
### La démarche de choix



### Approche Polyglot Persistence

Le **polyglot persistence** consiste à utiliser **plusieurs bases de données** dans la même application, chacune choisie pour ce qu'elle fait de mieux.

Exemple d'architecture d'un site e-commerce :



**Avantages** : chaque composant utilise l'outil le mieux adapté à ses besoins.

#### Inconvénients :

- Complexité opérationnelle accrue (maintenir plusieurs systèmes)
- Pas de transactions cross-bases
- Courbe d'apprentissage pour chaque technologie

---

## Notions d'architecture distribuée

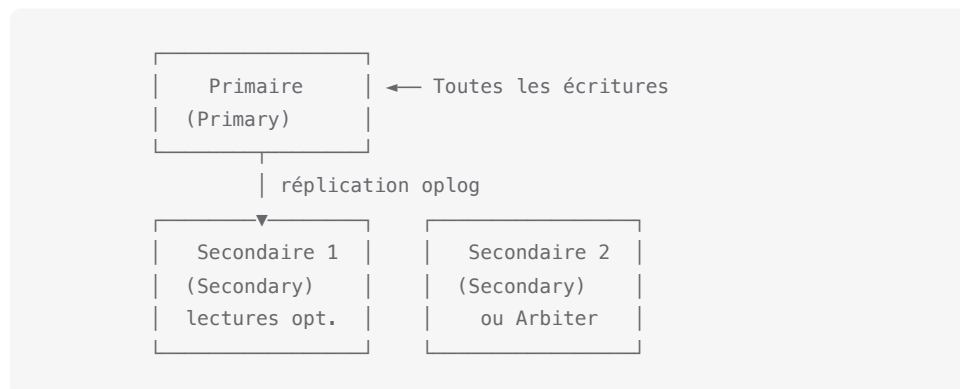
### La réplication

La **réplication** consiste à maintenir des **copies identiques** des données sur plusieurs serveurs (réplicas). Elle apporte :

- **Haute disponibilité** : si le serveur primaire tombe, un secondaire prend le relais
- **Performances en lecture** : les lectures peuvent être distribuées sur les secondaires
- **Durabilité** : les données sont en sécurité même en cas de panne matérielle

### Réplication dans MongoDB — Replica Set

Un **Replica Set** MongoDB est composé de :

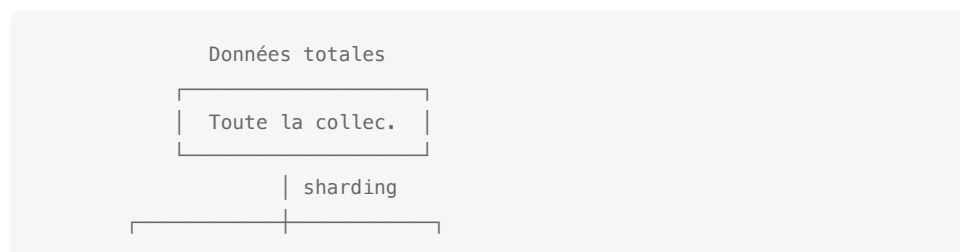


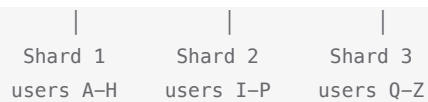
- Toutes les **écritures** se font sur le **primaire**
- Les modifications sont **répliquées** sur les secondaires via l'oplog
- En cas de panne du primaire, une **élection** désigne automatiquement un nouveau primaire
- Nombre minimum de nœuds recommandé : **3** (pour éviter le split-brain)

---

### Le sharding (partitionnement horizontal)

Le **sharding** consiste à **distribuer les données** sur plusieurs serveurs, chaque serveur ne stockant qu'une partie des données. C'est la solution pour dépasser les limites d'un seul serveur.





La **shard key** — clé de partitionnement

Le choix de la shard key est critique :

```
// Sharding de la collection "commandes" par clientId
sh.shardCollection("ecommerce.commandes", { clientId: "hashed" })
```

javascript

Une bonne shard key doit :

- Avoir une **cardinalité élevée** (beaucoup de valeurs distinctes)
- Distribuer les **écritures uniformément** (éviter les "hot spots")
- Permettre le **ciblage** : les requêtes fréquentes ne doivent interroger qu'un seul shard

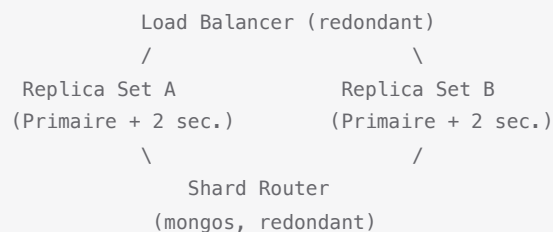
**Attention** : un mauvais choix de shard key est très difficile à corriger après coup.

## Haute disponibilité

La haute disponibilité (HA) vise à minimiser les interruptions de service. Elle repose sur plusieurs mécanismes :

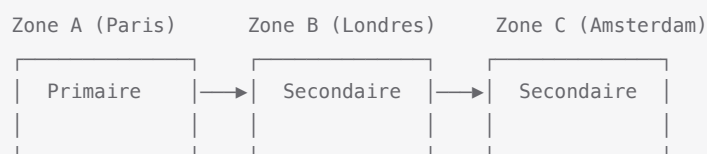
### Éliminer les points de défaillance unique (SPOF)

Chaque composant critique doit être redondant :



### Stratégie de déploiement multi-zones

Pour résister à la panne d'un datacenter entier, les réplicas doivent être dans des **zones de disponibilité** (Availability Zones) différentes, voire des régions géographiques différentes.



Si la Zone A tombe, le secondaire de la Zone B est élu primaire automatiquement.

## Résumé des concepts d'architecture distribuée

| Concept              | Rôle                                    | Technologie                        |
|----------------------|-----------------------------------------|------------------------------------|
| Réplication          | Copies des données, haute disponibilité | MongoDB Replica Set, Cassandra     |
| Sharding             | Distribution des données, scalabilité   | MongoDB Sharding, Cassandra vnodes |
| Load balancing       | Distribution du trafic                  | HAProxy, AWS ELB, mongos           |
| Consensus            | Élection du primaire                    | Raft (MongoDB), Paxos              |
| Cohérence éventuelle | Propagation asynchrone                  | Cassandra, DynamoDB                |

## Conclusion

Le paysage des bases de données s'est considérablement enrichi depuis l'émergence du NoSQL. Il n'existe pas de solution universelle : chaque famille répond à des besoins spécifiques.

**Les points clés à retenir :**

1. **Le théorème CAP** impose des compromis : on choisit entre cohérence forte et disponibilité maximale.
2. **MongoDB** est un excellent choix généraliste pour les données hétérogènes, avec une modélisation guidée par les requêtes.
3. **Redis** excelle comme couche de cache pour réduire la latence et soulager une base principale.
4. **Cassandra** est imbattable pour les charges massives en écriture et la distribution géographique.
5. **Neo4j** est le choix naturel quand les relations entre données sont aussi importantes que les données elles-mêmes.
6. Le **polyglot persistence** est une approche mature : combiner plusieurs technologies selon les besoins de chaque composant.
7. SQL et NoSQL ne s'opposent pas — ils se **complètent**. La question n'est pas "SQL ou NoSQL ?" mais "quel outil pour quel problème ?".



# Travaux pratiques MongoDB

Ces exercices s'appuient sur un scénario unique : une **plateforme de streaming vidéo**. Vous allez construire et interroger une base MongoDB composée de trois collections — `films`, `utilisateurs` et `visionnages` — pour mettre en pratique les opérations CRUD, les requêtes avancées, l'indexation et le pipeline d'agrégation.

---

## Mise en place

### Prérequis

Vous devez avoir NodeJS et Docker installé sur votre machine.

### Boilerplate

- Télécharger le projet [boilerplate-mongo](#) et l'extraire sur votre machine.
- Lancer le serveur mongo avec docker: `docker compose up -d`
- Installer les dépendences javascript: `npm i`
- Lancer le projet: `npm start`

#### Astuce

Creez un fichier `.js` par TP, et lancez-le avec `node TP1.js`.

---

## Rendu attendu

Pour chaque TP, vous devrez rendre un fichier javascript contenant les solutions aux exercices, séparés par des commentaires, par exemple:

```
// TP1.js

// Exercice 1.1
// a)
const f = films.find();
// b)
films.update();

// Exercice 2.1
// a)
films.insert();
// b)
const b = films.updateMany();
```

javascript

A la fin, créez une archive `.zip` avec chaque fichier `TPx.js` que vous devrez envoyer.

---

## TP 1 — Insertion et modélisation

### Seed

Lorsque vous démarrerez, la base de données est vide. Pour la peupler avec des données d'exemple, lancez la commande `node seed.js`.

Si vous voulez repartir sur une base vide, écrivez un script qui vide la base, ou alors supprimez et relancez le conteneur:

```
docker compose down
docker compose up -d
node seed.js
```

### Exercice 1.1 — Insérer des films

Insérez des films dans la collection `films`. Chaque document doit contenir :

- `titre` (string)
- `annee` (number)
- `genres` (tableau de strings)
- `realisateur` (string)
- `note` (number entre 0 et 10)
- `duree` (number, en minutes)
- `langues` (tableau de strings)

### Exercice 1.2 — Insérer les utilisateurs

Insérez les utilisateurs suivants dans une collection `utilisateurs`. Chaque document doit contenir :

- `nom` (string)
- `email` (string, unique)
- `age` (number)
- `abonnement` (string : `"basic"`, `"standard"` ou `"premium"`)
- `dateInscription` (date)
- `preferences` (objet avec un tableau `genres` et une liste de `realisateurs`)

### Exercice 1.3 — Insérer les visionnages

Récupérez les `_id` des films et utilisateurs que vous venez de créer (regarder ce que retourne `insert`), puis créez une collection `visionnages` pour enregistrer qui a regardé quoi.

La table `visionnages` doit contenir :

- `utilisateurId` (string)
- `filmId` (string)
- `date` (date)

- `noteUtilisateur` (number entre 0 et 10)
  - `termine` (boolean)
- 

## TP 2 — Requêtes et lecture

### Exercice 2.1 — Requêtes simples

- Trouvez tous les films avec une note supérieure ou égale à 8.7.
- Trouvez tous les films réalisés par Christopher Nolan.
- Trouvez les films sortis entre 2010 et 2015 inclus.

### Exercice 2.2 — Opérateurs logiques et tableaux

- Trouvez les films qui appartiennent au genre `"sci-fi"` ou `"comédie"`.
- Trouvez les films qui appartiennent à la fois au genre `"thriller"` et `"drame"`.
- Trouvez les films disponibles en langue `"français"`.
- Trouvez les films qui ont **exactement** 2 genres.

### Exercice 2.3 — Projections, tri et pagination

- Listez uniquement le titre et la note de tous les films (sans le champ `_id`).
- Affichez les 3 films les mieux notés (titre + note uniquement).
- Affichez les films triés par année décroissante, en affichant la page 2 avec 3 films par page.

### Exercice 2.4 — Documents imbriqués

- Trouvez les utilisateurs qui ont `"sci-fi"` dans leurs préférences de genres.
  - Trouvez les visionnages qui ont été terminés ( `termine: true` ) avec une note utilisateur supérieure ou égale à 9.
- 

## TP 3 — Mises à jour et suppressions

### Exercice 3.1 — Mises à jour simples

- La note du film "Oldboy" a été révisée à 8.5. Mettez-la à jour.
- Ajoutez le champ `streaming: true` à tous les films de Christopher Nolan.

### Exercice 3.2 — Opérateurs de mise à jour avancés

- Ajoutez le genre `"classique"` au film "Pulp Fiction" (sans dupliquer s'il existe déjà).

- b) Incrémentez de 1 le champ `likes` pour le film "Inception". Ce champ n'existe pas encore — il doit être créé automatiquement à 1.
- c) Retirez `"action"` des genres du film "The Dark Knight".

### Exercice 3.3 — Upsert et suppression

- a) Un film "Tenet" (2020, Christopher Nolan, note 7.4, durée 150 min) doit être ajouté s'il n'existe pas, ou mis à jour s'il existe déjà. Utilisez un `upsert`.
- b) Supprimez tous les visionnages non terminés ( `termine: false` ).
- c) Supprimez le champ `streaming` de tous les films où il existe.

---

## TP 4 — Indexation

### Exercice 4.1 — Créer des index adaptés

Pour chacune des requêtes suivantes, identifiez le ou les champs à indexer et créez l'index approprié.

- a) La requête la plus fréquente sur `films` est : trouver les films d'un réalisateur triés par note décroissante.
- b) Les visionnages sont souvent filtrés par `utilisateurId` pour afficher l'historique d'un utilisateur.

### Exercice 4.2 — Index et requêtes textuelles

- a) Créez un index textuel sur le champ `titre` de la collection `films`.
- b) Recherchez tous les films dont le titre contient le mot `"dark"` ou `"mad"`.
- c) Supprimez l'index textuel que vous venez de créer.

---

## TP 5 — Pipeline d'agrégation

### Exercice 5.1 — Jointures avec `$lookup`

- a) Pour chaque visionnage, récupérez le titre et la note du film correspondant (jointure `visionnages` → `films` ).
- b) Pour chaque visionnage terminé, affichez le nom de l'utilisateur et le titre du film.
- c) Calculez le nombre de visionnages par film, en affichant le titre du film. Triez par nombre de visionnages décroissant.

### Exercice 5.2 — Statistiques simples

- a) Calculez la note moyenne, maximale et minimale de tous les films.

- b) Comptez le nombre de films par réalisateur, triés par nombre de films décroissant.
- c) Calculez la durée totale et la durée moyenne des films par genre. (Indice : utilisez `$unwind` pour décomposer le tableau `genres`.)

### Exercice 5.3 — Filtrage dans le pipeline

- a) Listez les réalisateurs ayant au moins 2 films dans le catalogue, avec leur note moyenne.
- b) Trouvez les 3 genres les mieux notés en moyenne (parmi les genres ayant au moins 2 films).
- c) Calculez le nombre de visionnages terminés et non terminés, groupés par statut.

---

## Bonus: Performances

### Exercice 6.1 — Analyser les performances

- a) Avant de créer un index sur `note`, utilisez `explain("executionStats")` pour analyser le plan d'exécution de la requête suivante. Notez le champ `totalDocsExamined`.

```
db.films.find({ note: { $gte: 8.5 } }).explain("executionStats")
```

javascript

- b) Créez un index sur `note`, puis relancez la même requête avec `explain`. Comparez `totalDocsExamined` avant et après.
- c) Listez tous les index de la collection `films` avec `getIndexes()`.

### Exercice 6.2 — Statistiques croisées (synthèse)

- a) Quel utilisateur a la meilleure note moyenne dans ses visionnages terminés ? Affichez son nom et sa note moyenne.
- b) Pour chaque abonnement ( `basic`, `standard`, `premium` ), calculez : le nombre d'utilisateurs, le nombre total de visionnages et la note moyenne donnée.
- c) Construisez un rapport des films les plus populaires : pour chaque film, affichez le titre, la note officielle, le nombre de visionnages et la note moyenne donnée par les utilisateurs. Triez par note utilisateur décroissante.

# Bases de données orientées graphe

---

## Principe de fonctionnement

### Pourquoi les graphes ?

Certaines données sont fondamentalement relationnelles dans le sens *graphe* du terme : les connexions entre entités sont aussi importantes que les entités elles-mêmes.

### Le problème des jointures récursives en SQL

Prenons un réseau social. Pour trouver les amis d'amis d'une personne en SQL :

```
-- Amis directs
SELECT b.id FROM amis WHERE a_id = 1;

-- Amis d'amis : déjà plus complexe
SELECT c.id FROM amis b1
JOIN amis b2 ON b1.b_id = b2.a_id
WHERE b1.a_id = 1;

-- Amis d'amis d'amis : encore une jointure...
-- À N degrés de séparation : N jointures, performances catastrophiques
```

sql

Les bases de données relationnelles ne sont pas optimisées pour la traversée de relations profondément imbriquées. Plus le graphe est profond, plus les requêtes SQL deviennent lentes et complexes.

### La solution : stocker les relations comme des citoyens de première classe

Dans une base orientée graphe, les relations sont des entités à part entière avec leurs propres propriétés, stockées physiquement de façon adjacente aux nœuds qu'elles relient. Parcourir une relation est une opération à temps constant, quel que soit la taille du graphe.

---

## Les composants d'un graphe de propriétés

Le modèle le plus courant est le **Property Graph Model** (utilisé par Neo4j) :

### Les nœuds (Nodes)

Représentent les **entités** du domaine. Chaque nœud peut avoir :

- Un ou plusieurs **labels** (types) : `Person` , `Movie` , `City`
- Des **propriétés** (paires clé-valeur) : `{ nom: "Alice", age: 28 }`

```
(Alice:Person { nom: "Alice", age: 28 })
```

### Les relations (Relationships)

Représentent les **connexions** entre nœuds. Chaque relation a :

- Un **type** obligatoire : `CONNAÎT` , `TRAVAILLE_CHEZ` , `A_ACHETÉ`
- Une **direction** : de A vers B
- Des **propriétés** optionnelles : `{ depuis: 2020, force: 0.8 }`

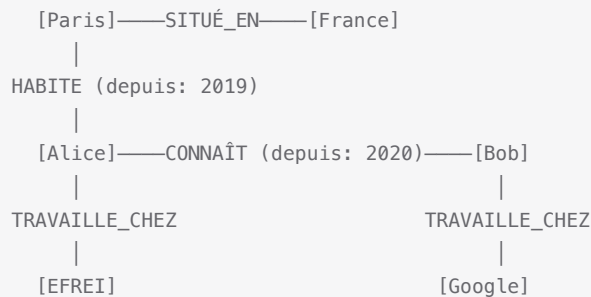
```
(Alice)-[:CONNAÎT { depuis: 2020 }]->(Bob)
```

Les relations sont **directionnelles** dans le modèle, mais peuvent être traversées dans les deux sens dans les requêtes.

## Les propriétés

Tant les nœuds que les relations peuvent stocker des propriétés (chaînes, nombres, booléens, listes...).

## Représentation visuelle



## Neo4j et le langage Cypher

### Neo4j

**Neo4j** est la base de données orientée graphe la plus utilisée au monde. Elle est disponible en version open source (Community Edition) et entreprise.

Caractéristiques techniques :

- Stockage natif des graphes (index-free adjacency)
- Transactions ACID complètes
- Interface web **Neo4j Browser** pour explorer les graphes visuellement
- Langage de requête **Cypher**
- API REST et drivers officiels (Python, JavaScript, Java, Go...)

### Le langage Cypher

**Cypher** est le langage de requête déclaratif de Neo4j. Sa syntaxe est conçue pour ressembler visuellement à des graphes.

## Syntaxe de base

```
-- Nœud avec label et propriétés
(alice:Person { nom: "Alice", age: 28 })

-- Relation directionnelle
(alice)-[:CONNAÎT]->(bob)

-- Relation avec propriétés
(alice)-[:CONNAÎT { depuis: 2020 }]->(bob)
```

cypher

## Créer des données — CREATE et MERGE

```
// Créer un nœud
CREATE (p:Person { nom: "Alice", age: 28 })

// Créer plusieurs nœuds et une relation
CREATE (alice:Person { nom: "Alice" })
CREATE (bob:Person { nom: "Bob" })
CREATE (alice)-[:CONNAÎT { depuis: 2020 }]->(bob)

// MERGE : créer si n'existe pas, sinon ne rien faire
MERGE (p:Person { nom: "Alice" })
ON CREATE SET p.createdAt = datetime()
ON MATCH SET p.lastSeen = datetime()
```

cypher

## Interroger les données — MATCH

```
// Trouver tous les nœuds Person
MATCH (p:Person)
RETURN p.nom, p.age

// Trouver les amis d'Alice
MATCH (alice:Person { nom: "Alice" })-[:CONNAÎT]->(ami)
RETURN ami.nom

// Trouver les amis d'amis d'Alice
MATCH (alice:Person { nom: "Alice" })-[:CONNAÎT*2]->(ami2)
RETURN DISTINCT ami2.nom

// Chemins de longueur variable (1 à 3 degrés de séparation)
MATCH (alice:Person { nom: "Alice" })-[:CONNAÎT*1..3]->(personne)
RETURN DISTINCT personne.nom
```

cypher

## Filtrer avec WHERE

```
// Personnes de plus de 25 ans qui connaissent Alice
MATCH (p:Person)-[:CONNAÎT]->(alice:Person { nom: "Alice" })
WHERE p.age > 25
```

cypher



```
RETURN p.nom, p.age
ORDER BY p.age DESC
```

## Mettre à jour — SET et REMOVE

```
// Ajouter/modifier une propriété
MATCH (p:Person { nom: "Alice" })
SET p.ville = "Paris", p.age = 29

// Supprimer une propriété
MATCH (p:Person { nom: "Alice" })
REMOVE p.age

// Ajouter un label
MATCH (p:Person { nom: "Alice" })
SET p:Admin
```

cypher

## Supprimer — DELETE et DETACH DELETE

```
// Supprimer une relation
MATCH (alice:Person { nom: "Alice" })-[r:CONNAÎT]->(bob:Person { nom: "Bob" })
DELETE r

// Supprimer un nœud et toutes ses relations
MATCH (p:Person { nom: "Alice" })
DETACH DELETE p
```

cypher

## Exemples avancés

Recommandation : films vus par des amis mais pas encore par Alice

```
MATCH (alice:Person { nom: "Alice" })-[r:CONNAÎT]->(ami)-[:A_VU]->(film:Movie)
WHERE NOT (alice)-[:A_VU]->(film)
RETURN film.titre, COUNT(ami) AS recommandations
ORDER BY recommandations DESC
LIMIT 5
```

cypher

Chemin le plus court entre deux personnes

```
MATCH path = shortestPath(
  (alice:Person { nom: "Alice" })-[r:CONNAÎT*]-(cible:Person { nom: "Charlie" })
)
RETURN path, length(path) AS distanceSociale
```

cypher

## Cas d'usage

Réseaux sociaux

Le cas d'usage originel des bases graphe. Les requêtes typiques sont :

- **Suggestions d'amis** : trouver les amis d'amis non encore connectés
- **Fil d'actualité** : trouver les publications des personnes que je suis
- **Degré de connexion** : combien de degrés séparent deux utilisateurs ?
- **Détection de communautés** : identifier des groupes de personnes fortement connectées

```
// Suggestion d'amis : amis d'amis pas encore connectés à Alice
MATCH (alice:Person { nom: "Alice" })-[:CONNAÎT]->(ami)-[:CONNAÎT]->(suggestion)
WHERE NOT (alice)-[:CONNAÎT]->(suggestion)
      AND suggestion <> alice
RETURN suggestion.nom, COUNT(ami) AS amisEnCommun
ORDER BY amisEnCommun DESC
```

cypher

## Détection de fraude

Les graphes excellent pour détecter des schémas frauduleux basés sur les relations entre entités :

- Plusieurs comptes bancaires reliés à un même numéro de téléphone
- Transactions circulaires entre des entités apparemment indépendantes
- Partage d'adresses IP, d'adresses postales ou de cartes bancaires entre des comptes "distincts"

```
// Détecter des comptes partageant le même numéro de téléphone
MATCH (c1:Compte)-[:UTILISE]->(tel:Telephone)<-[:UTILISE]-(c2:Compte)
WHERE c1 <> c2
RETURN c1.id, c2.id, tel.numero AS telephonePartagé
```

cypher

## Moteurs de recommandation

Les recommandations basées sur le comportement collaboratif sont naturellement exprimées en graphe :

```
// "Les clients qui ont acheté X ont aussi acheté..."
MATCH (client:Client)-[:A_ACHETÉ]->(produit:Produit { nom: "Laptop" })
      <-[:A_ACHETÉ]-(autreClient:Client)-[:A_ACHETÉ]->(recommandation:Produit)
WHERE NOT (client)-[:A_ACHETÉ]->(recommandation)
RETURN recommandation.nom, COUNT(autreClient) AS score
ORDER BY score DESC
LIMIT 5
```

cypher

## Autres cas d'usage

| Domaine       | Application                                              |
|---------------|----------------------------------------------------------|
| Logistique    | Optimisation de routes, graphes de transport             |
| Cybersécurité | Analyse de réseaux d'attaquants, propagation de malwares |

| Domaine                   | Application                                               |
|---------------------------|-----------------------------------------------------------|
| Sciences de la vie        | Réseaux de protéines, interactions médicamenteuses        |
| Gestion des connaissances | Knowledge graphs, ontologies, graphes de connaissances    |
| IT / Infrastructure       | Cartographie des dépendances, analyse d'impact            |
| Finance                   | Analyse des risques de contrepartie, chaînes de propriété |

---

## Quand choisir une base orientée graphe ?

### Choisir Neo4j si :

- Les **relations entre entités** sont au cœur du modèle métier
- Vous avez besoin de **traversées profondes** (plusieurs degrés de séparation)
- Les requêtes portent sur des **patterns de connexions** complexes
- La structure du graphe **évolue fréquemment**

### Ne pas choisir Neo4j si :

- Les données sont principalement tabulaires sans relations complexes
- Les requêtes sont essentiellement des lectures simples par clé
- Vous avez besoin de traiter des volumes massifs de données analytiques (préférer Cassandra ou un data warehouse)

Partie 1 — Introduction et fondements (1h30) 1.1 Rappels sur les bases de données relationnelles et leurs limites face aux besoins actuels (volumétrie, scalabilité horizontale, données non structurées) 1.2 Contexte d'émergence du NoSQL : Big Data, applications web à grande échelle 1.3 Le théorème CAP et le modèle BASE vs ACID 1.4 Panorama des quatre familles NoSQL : clé-valeur (Redis), document (MongoDB), colonne (Cassandra), graphe (Neo4j) — principes, cas d'usage et positionnement de chacune

Partie 2 — MongoDB : concepts et modélisation (1h30) 2.1 Architecture de MongoDB : bases, collections, documents JSON/BSON, ObjectId 2.2 Schéma flexible : avantages, risques et bonnes pratiques de validation de schéma 2.3 Modélisation documentaire : imbrication vs référencement, dénormalisation assumée 2.4 Penser par les requêtes : concevoir son modèle en fonction des accès aux données 2.5 Comparaison avec la modélisation relationnelle sur un exemple concret

Partie 3 — MongoDB : requêtes et manipulation de données (1h30) 3.1 Opérations CRUD : insertOne, insertMany, find, updateOne, updateMany, deleteOne, deleteMany 3.2 Opérateurs de requête : comparaison (gt, \$lt, \$in...), logiques ( and, or, \$not), sur tableaux ( elemMatch, \$all, \$size)

3.3 Requetes sur documents imbriqués et notation pointée 3.4 Projections, tri et pagination (sort, skip, limit) 3.5 Mises à jour partielles : \$set, \$unset, \$push, \$pull, \$inc, \$addToSet, upsert

Partie 4 — MongoDB : agrégation et indexation (1h) 4.1 Pipeline d'agrégation : principes et étapes principales (\$match, \$group, \$sort, \$project, \$unwind, \$lookup) 4.2 Exemples concrets : statistiques, jointures entre collections, transformations de données 4.3 Indexation : index simples, composés, textuels, multiclés 4.4 Analyser les performances d'une requête avec explain()

Partie 5 — Bases de données orientées graphe (0h30) 5.1 Principe de fonctionnement : nœuds, relations, propriétés 5.2 Présentation de Neo4j et du langage Cypher 5.3 Cas d'usage : réseaux sociaux, détection de fraude, recommandation

Partie 6 — Synthèse et choix technologiques (1h) 6.1 Tableau comparatif des familles NoSQL : modèle de données, cohérence, scalabilité 6.2 NoSQL vs SQL : quand choisir l'un ou l'autre, approche polyglot persistence 6.3 Notions d'architecture distribuée : réplication, sharding, haute disponibilité